A programozás alapjai 1 - BMEVIHIAA01 2025/26/1

Ballisztikai számítási program

Készítette: Horváth Ádám József,
M74IPN
horvadam06@gmail.com

Tartalomjegyzék

A programozás alapjai 1 - BMEVIHIAA01 2025/26/1 1
1. Specifikáció 3
1.1.0 A program célja 3
1.2.0 Bemenő adatok és típusai 3
1.3.0. Program működése 4
1.4.0 Kimeneti adatok 5
1.5.0 Adatszerkezetek 6
1.6.0 Tesztelés 6
2. Adatszerkezetek 7
2.1.0 Changelog 7
2.2.0 Adatkezelés 8
2.2.1 Láncolt strázsált lista 8
2.3.0 Löveg lista 9
2.3.1 Löveg adatszerkezet 9
2.3.2 Löveg adatszerkezet főbb függvényei 10
2.4.0 Bázisok, Célpontok 11
2.4.1 Alap pozíció adatok 11
2.4.2 Bázis lista 11
2.4.3 Célpontok 13
2.4.4 Célpontok adatszerkezetek főbb függvényei 14
2.5.0 Program flowchart 17
2.5.1 Rendszerindítás és inicializáció 18
2.5.2 Konfiguráció betöltése 18
2.5.3 Parancsértelmezés és fő vezérlési ciklus 18
2.5.4 Adatszerkezetek kezelése 18
2.5.5. Kilépés és memóriafelszabadítás 19

1. Specifikáció

1.1.0 A program célja

A program célja ballisztikai számítások elvégzése különböző lőállások, célpontok és lövegek adatainak felhasználásával.
Feladata, hogy meghatározza, melyik lőállás van a legközelebb a kijelölt célponthoz, majd kiszámolja a szükséges lövési szöget és irányt a löveg paramétereinek figyelembevételével.

A felhasználó megadhatja a célpontot, opcionálisan a lőállást és a löveg típusát is. Ha ezek hiányoznak, a program automatikusan kiválasztja a legideálisabb kombinációt.
Amennyiben a megadott löveg vagy lőállás nem képes elérni a célpontot, a program ajánl egy paramétereiben hasonló, de alkalmasabb megoldást.
Ha ilyen sincs, új lőállás vagy célpont rögzíthető és eltárolható.

1.2.0 Bemenő adatok és típusai

A bemeneti adatok szöveges fájl(ok)ból származnak.
A program a fájlokat egyszeri bejárással olvassa be, és a tartalmat dinamikus adatszerkezetekbe rendezi.
A rekordok formátuma rugalmas, a kulcsszavak sorrendje tetszőleges, viszont minden sor “;” jellel zárul.

1.2.1. Általános formátum

I<ID> L<Label> CLA<Latitude> CLO<Longitude> CA<Altitude>;

1.2.2. Példa

I0 LSchonherz CLA47.4729283 CLO19.0531429 CA0.0;

I1 LValami CLA57.476002 CLO29.070322 CA2.2;

1.2.3. Szabályok

// utáni sorokat a program figyelmen kívül hagyja.

/* ... */ közötti blokkokat szintén kihagyja.

A CLA, CLO, CA értékei lebegőpontos számok.

Az I azonosítók normalizálva lesznek, ha duplikátum előfordul.

Az adatok sorrendje tetszőleges, a parser a kulcsok alapján dolgozik.

1.2.4. További fájlok

A program más típusú adatokat is kezelhet, például löveg paramétereket:

G0 LHE EXP_RAD10 MASS50;

G1 LNUC EXP_RAD500 MASS400;

1.3.0. Program működése

1.3.1. Beolvasás

A program beolvassa az adatokat, majd dinamikus adatszerkezetet épít a memóriában:

- lőállások listája (koordináták, lövegkészlet),
- célpontok listája (koordináták, méret),
- lövegek listája (típus, tömeg, hatótáv, robbanási sugár, stb.).

1.3.2. Kapcsolatok felépítése

A program minden célponthoz kiszámolja, melyik lőállás van a legközelebb.
Ez alapján létrejön a célpont–lőállás párosítás.

1.3.3. Számítás

A kiválasztott vagy automatikusan választott löveg alapján történik a tényleges számítás:

- távolság a célpont és lőállás között,
- szükséges lövési szög,
- irány,
- energia és hatótáv kiértékelés.

1.3.4. Ajánlás és döntés

A program elemzi a löveg és a célpont paramétereit:

- ha a löveg túl erős, olcsóbb alternatívát ajánl,
- ha több kisebb löveg hatékonyabb, kombinált megoldást javasol,
- ha a célpont elérhetetlen, új lőállást vagy löveget ajánl.

1.4.0 Kimeneti adatok

A számítás eredményeit a program **alapból nem menti**, csak megjeleníti.
A felhasználó választhat, hogy szeretné-e menteni az eredményeket.

1.4.1. Mentési lehetőségek

Új fájlba mentés: a felhasználó megadja a fájlnevet.

Felülírás: ha a fájl már létezik, rákérdez, hogy felülírja-e.

Hozzáfűzés: lehetőség van az új adatok hozzáírására a fájl végéhez.

1.4.2. Kimeneti formátum

2000.01.01. 00:00:00----------

Löveg: 120mm_HE

Célpont: LTeSt_DaTa1

Lőállás: LSchonherz

Távolság: 2050.42 m

Szög: 35.8°

Irány: 78.3°

Javaslat: Megfelelő

1.5.0 Adatszerkezetek

A program a beolvasott adatokat láncolt listákban tárolja, az adatok közti kapcsolatokat pointerekkel oldja meg.
Ez biztosítja, hogy az adatok ne legyenek többször tárolva, és az algoritmus hatékonyan működjön.
A program kilépéskor felszabadítja a lefoglalt memóriát, hogy ne legyen memória-szivárgás.

1.6.0 Tesztelés

A program működésének ellenőrzésére külön tesztfájlok készültek.
Példa tesztadat:

```
I0 LTeSt_DaTa0 CLA40.05 CLO20.0 CA-5.023044;
CLO20.0 CA-5.023044 I1 CLA40.05 LTeSt_DaTa1;
```

Ezek a fájlok ellenőrzik:

- hibás vagy kevert sorrendű bemenetek kezelését,
- kommentek figyelmen kívül hagyását,
- duplikált azonosítók javítását,
- számítások helyességét és ajánlási logikát.

1.6.1 Interpreter teszt

- Teszt dokumentum:

```
//------------Input test--------------
//this is a comment
/*
this is a comment too
I0 LTeSt_DaTa CLA40.05 CLO20.0 CA0-5.023044;
*/
I0 LTeSt_DaTa0 CLA40.05 CLO20.0 CA0-5.023044;
CLO20.0 CA0-5.023044        I1 CLA40.05     LTeSt_DaTa1;
```

- Valós eredmény:

```
Starting test...
[Line comment] :
----------Input test----------
[Line comment] :
this is a comment
[Block comment] :
this is a comment too

I0 LTeSt_DaTa CLA40.05 CLO20.0 CA0-5.023044;
capacity: 10, items: 2
0 TeSt_DaTa0
1 TeSt_DaTa1
---------------------------------
Test succeeded!
---------------------------------
```

---

2. Adatszerkezetek

2.1.0 Changelog

A programot kibővítettem egy automatikusan generált config.txt file-al, valamint opcionálisan színes karakter megjelenítéssel (windows rendszeren szükséges hozzá a

„reg add HKCU\\Console /v VirtualTerminalLevel /t REG_DWORD /d 1 /f”

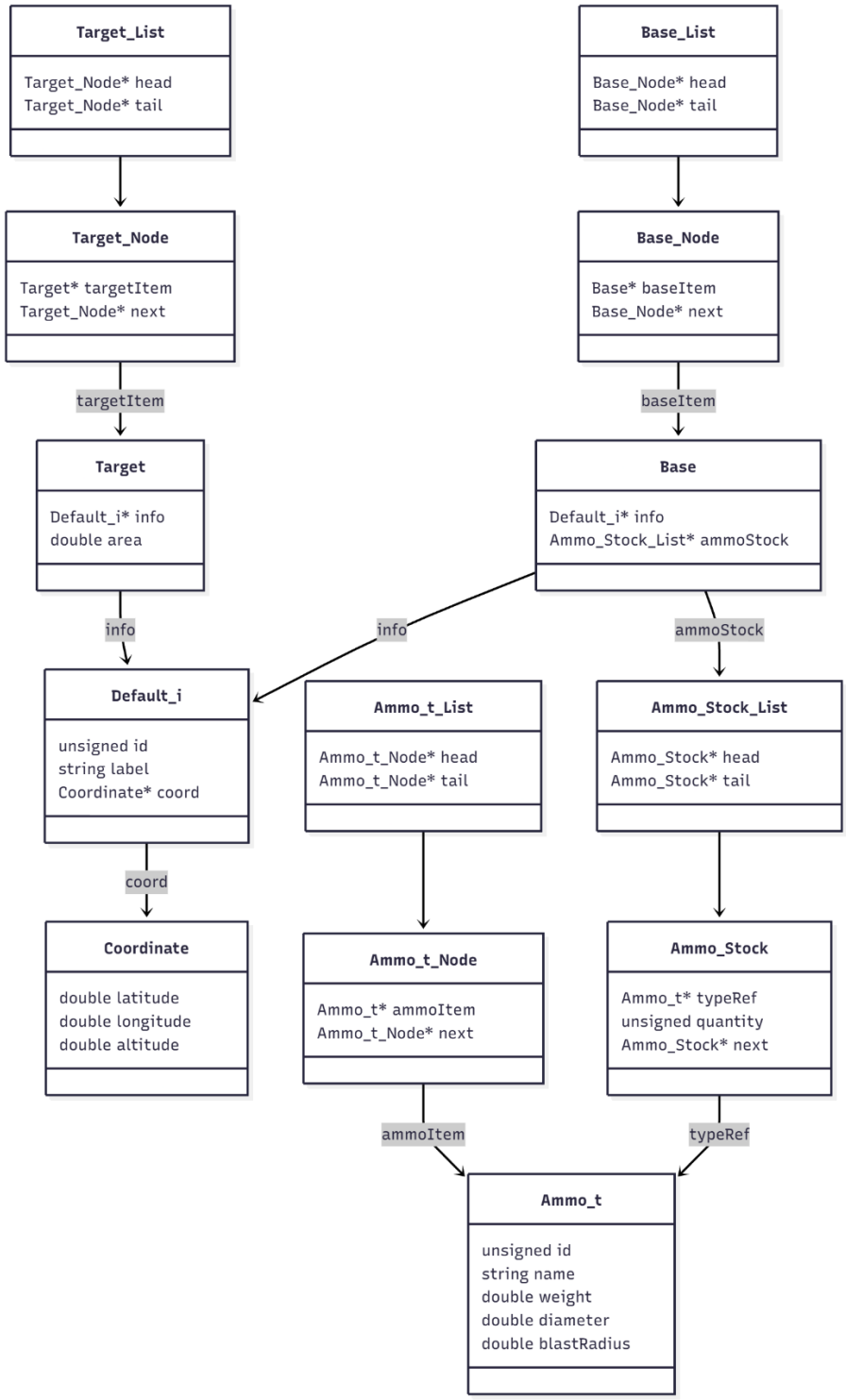
futtatással, mivel a Windows terminál alapból nem támogatja az ANSI escape kódokat, csak virtuális módban), a jobb felhasználói élmény érdekében a CLI parancskészlet bővült, valamint modal loop támogatást kapott. Újra dolgoztam a file betöltő praseret, interpretert, valamint tokenizert. A config file-nak köszönhetően az adat listáknak most már nem kell a munka könyvtárban lenniük.

2.2.0 Adatkezelés

2.2.1 Láncolt strázsált lista

Az adatok tárolása 4 elől-hátul strázsált, hátrafelé láncolt listában történik. Ezek közül 2 fésűs elrendezésben van.

[HEAD] => [elem1] => [elem2] => ... => [TAIL]



2.3.0 Löveg lista

A löveglista (Ammo_t_List) a program globális lőszer-adatbázisa.
Ez tartalmazza a rendszerben elérhető összes lőszer típus definícióját, amelyeket később a bázisok készletei (Ammo_Stock) referencia alapján hivatkoznak.

A lista ID szerint rendezett, elöl-hátul strázsás láncolt lista.

A globális lőszeradatok a következő információkat tartalmazzák:

- egyedi azonosító
- név
- fizikai paraméterek (tömeg, átmérő, robbanási sugar)

2.3.1 Löveg adatszerkezet

- **Ammo_t** - egyetlen lőszer definíció. Egy lőszer fizikai és azonosítási adatait tartalmazza.
- **Ammo_t_Node** - A lista egy eleme, és egy pointerrel tovább kapcsolódik a listába.
- **Ammo_t_List** - globális lőszerlista, ID-k alapján sorrendben tárolja a típusokat

```
/*Global Ammo type*/
typedef struct Ammo_t {
    unsigned id;
    char* name;         // dynamically allocated string
    double weight;
    double diameter;
    double blastRadius;
} Ammo_t;

/*Ammo_t_Node for linked list of global ammo*/
typedef struct Ammo_t_Node {
    Ammo_t* ammoItem;
    struct Ammo_t_Node* next;
} Ammo_t_Node;

/*Global Ammo list*/
typedef struct {
    Ammo_t_Node* head;
    Ammo_t_Node* tail;
} Ammo_t_List;
```

2.3.2 Löveg adatszerkezet főbb függvényei

```
// --- Ammo List, ordered by id ---
void initAmmo_t_List(Ammo_t_List* list);
Ammo_t_Node* addAmmo_t(Ammo_t_List* list, Ammo_t* ammo);
int deleteAmmo_t(Ammo_t_List* list, unsigned id);
Ammo_t_Node* popFrontAmmo_t(Ammo_t_List* list);
Ammo_t_Node* popBackAmmo_t(Ammo_t_List* list);
void freeAmmo_t_List(Ammo_t_List* list);
```

initAmmo_t_List

- üres listát készít
- head = tail = NULL
- a globális ammo-lista létrehozásakor hívódik meg

addAmmo_t

- új lőszert ad a listához
- ID szerint rendezett beszúrás
- visszatér az új csomópont pointerével

deleteAmmo_t

- töröl egy lőszert azonosító alapján
- ha nem létezik → 0
- ha sikerült → 1

popFrontAmmo_t / popBackAmmo_t

- sor-szerű vagy verem-szerű felhasználáshoz
- elsősorban CLI parancsokhoz vagy teszteléshez

freeAmmo_t_List

- felszabadítja az összes csomópontot és a dinamikus neveket

2.4.0 Bázisok, Célpontok

A bázisok és a célpontok közös alapja a Default_i struktúra, amely egységesíti az azonosítást és a koordinátakezelést.
A differenciák a bázis-specifikus és célpont-specifikus adatmezőkben jelennek meg.

2.4.1 Alap pozíció adatok

Default_i - egységes pozíció adatszerkezet, általános paraméter adatokat tartalmaz

```
/*Default_i (shared by Base and Target)*/
typedef struct {
    unsigned id;
    char* label;        // dynamically allocated string
    Coordinate* coord;  // pointer to Coordinate
} Default_i;
```

Szerepe:

Egy egységes struktúra meghatározása:

- egyedi azonosító
- név/jelző
- pozíció (Coordinate)
- duplikáció minimalizálása

2.4.2 Bázis lista

Ammo_Stock - bázisokra jellemző lőszerkészlet, Ammo_t * tartalmaz

```
/*Base-specific Ammo Stock*/
typedef struct Ammo_Stock {
    Ammo_t* typeRef;           // pointer to global ammo
    unsigned quantity;
    struct Ammo_Stock* next;
} Ammo_Stock;
```

Szerepe:

- A bázison tárolt lőszerek mennyiségét reprezentálja.

Ammo_Stock_List - strázsás lista lőszerkészletre. Minden bázis saját, külön lőszerkészletet tart fent.

```
/*Ammo stock list for each Base*/
typedef struct {
    Ammo_Stock* head;
    Ammo_Stock* tail;
} Ammo_Stock_List;
```

Base - egyetlen bázis adatai

```
/*Base struct*/
typedef struct Base {
    Default_i* info;
    Ammo_Stock_List* ammoStock;
} Base;
```

A bázis:

- rendelkezik pozícióval és névvel (info)
- saját lőszerraktárral (ammoStock)

Base_Node - A lista egy eleme

```
/*Base_Node for linked list*/
typedef struct Base_Node {
    Base* baseItem;
    struct Base_Node* next;
} Base_Node;
```

Base_List — bázisok tárolása

```
typedef struct {
    Base_Node* head;
    Base_Node* tail;
} Base_List;
```

Szerepe:

- A program futása alatt a bázis tárolására
- Futási idő végén a statikus lista módosítására szolgál

2.4.3 Célpontok

A célpontok szerkezete egyszerűbb, mert nem rendelkeznek készlettel és dinamikus tulajdonságokkal.

Target - egy célpont adatai

```
/*Target struct*/
typedef struct Target {
    Default_i* info;
    double area;
} Target;
```

A célpont:

- rendelkezik pozícióval (info->coord)
- rendelkezik megsemmisítendő területtel (area)
- a ballisztikai számítások ezt használják a szükséges lőirány meghatározásához

Target_Node - A lista egy eleme

```
/*Target_Node for linked list*/
typedef struct Target_Node {
    Target* targetItem;
    struct Target_Node* next;
} Target_Node;
```

Target_List - A célpont lista

```
typedef struct {
    Target_Node* head;
    Target_Node* tail;
} Target_List;
```

A célpontlista:

- betöltéskor feltöltődik
- CLI alatt módosítható (insert/delete)
- select_target innen választ
- bulk mód itt iterál végig ID szerint

2.4.4 Célpontok adatszerkezetek főbb függvényei

```
// --- Base List, ordered by id ---
void initBase_List(Base_List* list);
Base_Node* addBase(Base_List* list, Base* base);
int deleteBase(Base_List* list, unsigned id);
Base_Node* popFrontBase(Base_List* list);
Base_Node* popBackBase(Base_List* list);
void freeBase_List(Base_List* list);

// --- Ammo Stock List (per Base), ordered by ammo id ---
void initAmmo_Stock_List(Ammo_Stock_List* list);
Ammo_Stock* addAmmo_Stock(Ammo_Stock_List* list, Ammo_t* ammoRef,
unsigned quantity);
int deleteAmmo_Stock(Ammo_Stock_List* list, Ammo_t* ammoRef);
Ammo_Stock* popFrontAmmo_Stock(Ammo_Stock_List* list);
Ammo_Stock* popBackAmmo_Stock(Ammo_Stock_List* list);
void freeAmmo_Stock_List(Ammo_Stock_List* list);

// --- Target List, ordered by id ---
void initTarget_List(Target_List* list);
Target_Node* addTarget(Target_List* list, Target* target
int deleteTarget(Target_List* list, unsigned id);
Target_Node* popFrontTarget(Target_List* list);
Target_Node* popBackTarget(Target_List* list);
void freeTarget_List(Target_List* list);
```

- **initBase_List(Base_List* list)**

  A lista inicializálása.

  - Üres állapotba teszi (head, tail NULL, size = 0).
  - Minden további művelet csak ezután használható biztonságosan.

- **addBase(Base_List* list, Base* base)**

  Új bázis beszúrása rendezett helyre, tehát:

  - megkeresi azt a pozíciót, ahol az ID alapján lennie kell
  - létrehoz egy új csomópontot
  - beilleszti úgy, hogy a lista továbbra is rendezett marad

  Visszatérési érték: a létrehozott csomópont pointere.

- **deleteBase(Base_List* list, unsigned id)**

  Egy bázis törlése id alapján:

  - végigmegy a listán
  - ha talál pontos ID-t, kiveszi a csomópontot
  - felszabadítja
  - visszatér 0/1 vagy error kóddal

- **popFrontBase(Base_List* list)**

  Az első elemet kiveszi a listából:

  - head => head->next
  - leválasztja a régi head-et
  - visszaadja a kicsipkedett csomópontot (caller feladata felszabadítani)

  Tipikusan akkor kell, ha fájlból töltve végig akarod pucolni a listát.

- **popBackBase(Base_List* list)**

  Az utolsó elemet veszi ki:

  - végigmegy a lista végére
  - leválasztja a tail-t
  - visszaadja a csomópontot

  Ez lassabb egy sima egyszerű listában, de néha szükséges.

- **freeBase_List(Base_List* list)**

  Az egész lista felszabadítása:

  - végigmegy a csomópontokon
  - free() minden Base-re és node-ra
  - végül üres állapotba állítja

  Ez a helyes lezárás a program végén.

- **initAmmo_Stock_List**

  Üresre állítja a készletlistát.

- **addAmmo_Stock**

  Hozzáad egy új ammo tételt a megfelelő helyre, rendezve.
  Mivel ammoRef az ammo definíció pointere, a lista nem tárol másolatot, csak hivatkozást + quantity értéket.

- **deleteAmmo_Stock**

  Eltávolít egy adott ammo fajtát id alapján.
  Ez valójában az ammoRef->id szerint működik.

- **popFrontAmmo_Stock / popBackAmmo_Stock**

  Ugyanaz, mint a Base esetében - csomópontot kicsip, visszadobja a hívónak.

- **freeAmmo_Stock_List**

  Törli az adott Base saját ammo-készletét.

- **initTarget_List**

  Üres lista létrehozása.

- **addTarget**

  Új célpont hozzáadása id szerinti sorrendben.
  Ez fontos a gyorsabb keresés, átláthatóság és stabil output miatt.

- **deleteTarget**

  Célpont törlése id alapján.

- **popFrontTarget / popBackTarget**

  Az első / utolsó célpont kicsipése.

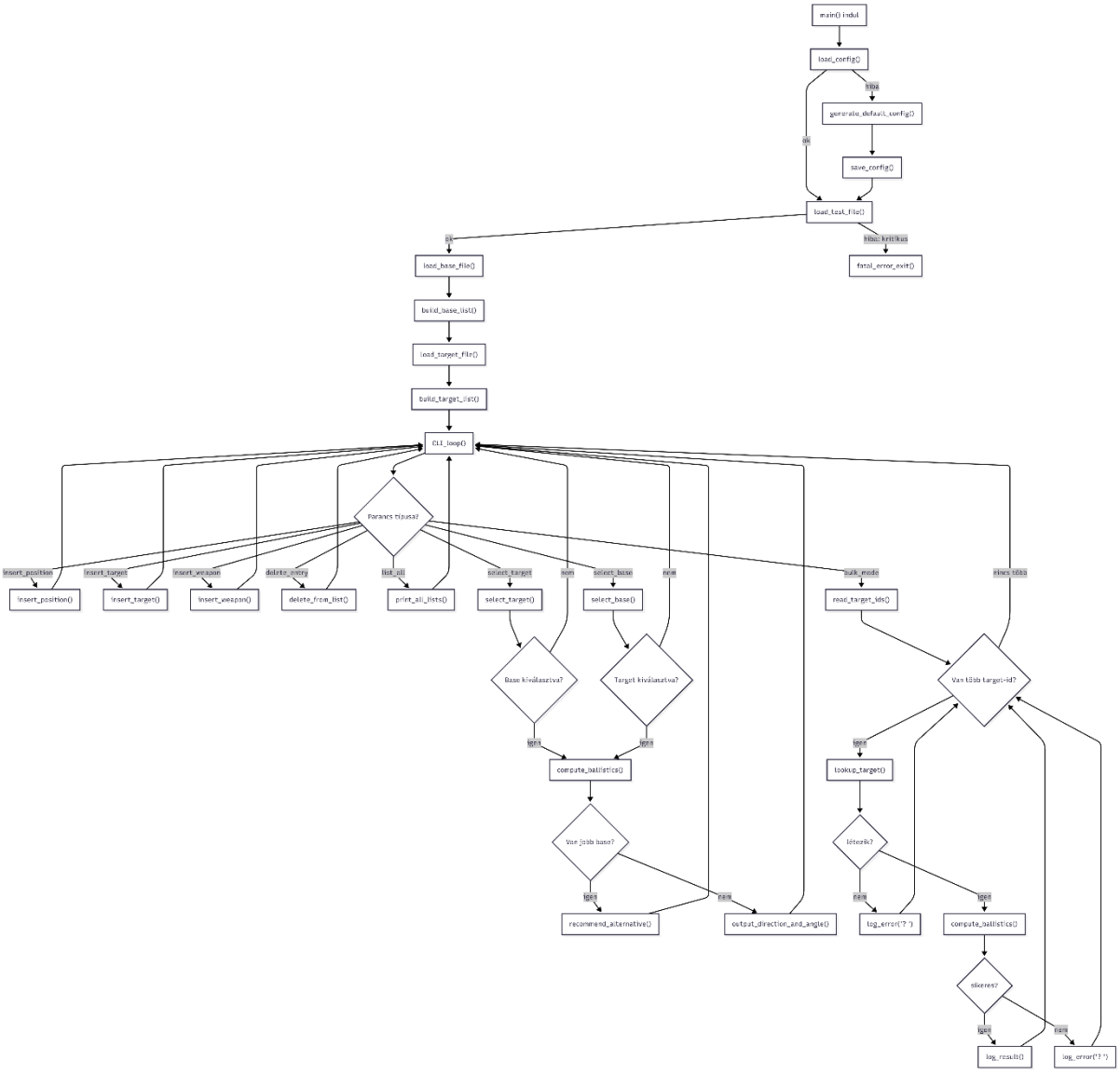
- **freeTarget_List**

  Az összes célpont törlése.

2.5.0 Program flowchart

A program négy fő lépésből áll:

- Inicializáció és rendszerelőkészítés
- Konfiguráció betöltése vagy létrehozása
- Parancsértelmezés és adatkezelés
- Tiszta kilépés és memóriafelszabadítás



2.5.1 Rendszerindítás és inicializáció

A program indulásakor az első feladat a rendszer előkészítése. Ez azt jelenti, hogy létrehozzuk a szükséges adatszerkezeteket, lefoglaljuk a memóriát, és alaphelyzetbe állítjuk az állapotváltozókat.
Ez biztosítja, hogy a bázislista, a lőszerkészletek és a célpontlista mindig következetes állapotban legyenek.

Ez az első lépés minden további funkcióhoz szükséges, és garantálja, hogy a program mindig ugyanúgy induljon el.

2.5.2 Konfiguráció betöltése

Ezután a program megpróbálja betölteni a korábbi beállításokat a konfigurációs fájlból.

- Ha sikerül: a program a fájlban található adatokkal folytatja a működést.
- Ha nem sikerül: a program létrehoz egy új, alapértelmezett konfigurációt. Ilyenkor a felhasználó dönthet, hogy menti-e az új beállításokat, vagy ideiglenesen futtatja a programot a létrehozott értékekkel.

Így mindig lesz használható konfiguráció, akár új, akár meglévő adatok alapján.

2.5.3 Parancsértelmezés és fő vezérlési ciklus

A program központi része a folyamatos parancsértelmező ciklus, ami a felhasználói vagy rendszer által indított parancsokat kezeli.

A folyamat:

- **Parancs beolvasása** - a felhasználó vagy a program indítja.
- **Érvényesség ellenőrzése** – a parancs helyes-e formailag és tartalmilag.
- **Parancsfeldolgozás** - érvényes parancsok meghívják a megfelelő modulokat, például: új bázis hozzáadása, bázis törlése, lőszerkészlet módosítása, célpontok kezelése, listák kiíratása.
- **Hibakezelés** - érvénytelen parancs esetén a program jelzi a hibát és új bemenetet kér.

A ciklus addig fut, amíg a felhasználó ki nem adja a kilépés parancsot.

2.5.4 Adatszerkezetek kezelése

- A program 4 listát kezel: bázisok és lőszerkészletek, célpontok, lőszer specifikációk.
- Az adatszerkezetek külön működnek, de a vezérlés egységesen történik.

2.5.5. Kilépés és memóriafelszabadítás

A kilépéskor a program tisztán zárja le a működését:

- Minden lista felszabadítása
- Opcionális konfiguráció mentése
- Erőforrások lezárása

Ez megakadályozza, hogy a program memóriaszivárgást okozzon.